Routed Recall Does Not Degrade at a Trillion Rows [Experiment, Analysis & Benchmark]

Routed search over compressed vectors, measured against the exact scan, on a shared research cluster

Andrew H. Bond
San José State University
San José, California, USA
andrew.bond@sjsu.edu

ABSTRACT

We report a measurement of routed inverted-file search over an index of one trillion 4-bit quantized vectors, built and queried on the National Research Platform's shared Kubernetes cluster. The index is 500 servers of two billion rows each, 24.0 bytes per row, 24 TB in all, on one block volume per server. Against the exact full scan of the same compressed index, routed search returns recall at ten of 0.989 when it probes 32 of 2048 cells and 0.999 when it probes 128, on 100 queries, with 95 percent bootstrap intervals of 0.982 to 0.995 and 0.997 to 1.000. The same protocol gave 0.979 and 0.999 at one hundred million rows, 0.982 and 0.999 at ten billion, and 0.983 and 0.999 at one hundred billion. Recall against the full scan does not degrade across four orders of magnitude of corpus size. Inside the trillion-row index, where subsets of servers differ only in the number of rows, recall moves by less than 0.01 at 128 probes and rises with the number of rows at 16 and 32 probes. Every scale point draws its queries by the same recipe, the first rows of four seeded shards spread across the index, so the comparison across sizes holds the query distribution fixed. That recipe also makes every query a corpus row, with a sixth of the reference neighbours on one server. A second set of 100 queries drawn under seeds no corpus shard uses, so that none is in the index and their neighbours spread over 404 servers, gave 0.969 at 32 probes and 0.999 at 128, with intervals of 0.954 to 0.982 and 0.997 to 1.000. The second result is that recall on the corpus-row queries is a property of the router alone. From the cell of each reference neighbour in its query's probe order we predicted recall at nine probe widths without a scan, the prediction matched the two widths already measured to the last digit, and a sweep at three further widths, run after the prediction was recorded, returned 0.952, 0.997 and 1.000, each equal to its predicted value. The measure is self-consistency of the compressed index on a synthetic corpus of intrinsic dimension 16, and the paper gives the float ground truth and real-embedding figures that bound it. Reranking each shortlist against float rows regenerated from the seed shows that the compressed ranking keeps at most 0.621 of the true neighbours on this corpus, and after that rerank the routed top ten agrees with the reranked reference at 0.982 for 32 probes and 1.000 from 128 on. The build and the job design that made the measurement possible on a shared cluster, almost entirely in its smallest job class, are described in a companion paper. Every log, state file, partial result and score is in the repository, and the identity and the bound the analysis rests on are checked in Lean 4 with Mathlib.

PVLDB Reference Format:

Andrew H. Bond. Routed Recall Does Not Degrade at a Trillion Rows [Experiment, Analysis & Benchmark]. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://doi.org/10.5281/zenodo.23608470.

1 INTRODUCTION

Approximate nearest-neighbour search over compressed vectors is routine at a billion rows [4, 10, 12, 18]. The question this paper answers is narrower than most. When an inverted-file index is partitioned into shards that share one coarse quantizer and one rotation, and the corpus grows by adding shards rather than by densifying old ones, does the routing lose anything it would have found by scanning everything? The way to ask it is to build the whole index, scan the whole index exactly, and compare. We did that at one trillion rows.

The index was built and measured on the National Research Platform's Nautilus cluster [20], a Kubernetes federation shared by many research groups across sites. A fleet measurement there is a matter of fair use by design. Jobs are small, independent and resumable, submissions are paced by a controller that yields when the cluster is busy, and nothing is reserved. That design, the seeded corpus that made the build move no data, and the pool that ran the measurement are the subject of a companion paper [3].

The paper makes two claims and states one limit. Routed recall against the exact scan of the same index is 0.989 at 32 probes and 0.999 at 128 probes at 10^{12} rows, with 95 percent bootstrap intervals over the 100 queries of 0.982 to 0.995 and 0.997 to 1.000, and the same protocol at 10^8, 10^{10} and 10^{11} rows gave figures within 0.01 of these at 32 probes and within 0.001 at 128 probes (Section 4). Inside the trillion-row index, where subsets of servers differ only in the number of rows, recall moves by less than 0.01 at 128 probes and rises with the number of rows at 16 and 32 probes (Section 4.2). The queries at every scale point are the first rows of four seeded shards spread across the index, so the trend across sizes is measured with the query recipe fixed, and Section 4.4 states what that recipe does to the placement of the reference neighbours and gives the recall

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

for 100 queries the index does not contain, 0.969 at 32 probes and 0.999 at 128. Recall at any width is the share of reference neighbours whose cell the router reaches, which was computed from the index metadata alone, matched the measured widths, and predicted three further widths before they were measured (Section 4.3). The limit is that the corpus is synthetic, of intrinsic dimension about 16, and the recall reported is against the compressed index's own exact scan rather than against float ground truth. Section 5 gives the figures that bound the result, including a real-embedding pilot on which the routing holds and the compression alone does not.

2 THE INDEX AND THE CORPUS

Index. Each row is a 32-dimensional vector projected to 24 dimensions by one basis shared by the whole fleet, quantized to 4 bits per dimension, and assigned to one of 2048 inverted-file cells by one shared coarse quantizer. Both the basis and the quantizer are fitted once, on a bootstrap shard, and read by every build job. A shard holds five million rows. A server holds 400 shards, two billion rows, on one block volume. The on-disk row is 12 bytes of codes, 8 bytes of per-row norms used by the asymmetric distance computation, and 4 bytes of cell membership. Every server reports 48 009 656 624 index bytes for its two billion rows, 24.0 bytes per row, and the fleet holds 24.0 TB of index on 500 volumes. Scoring is asymmetric distance computation over the codes, in row blocks, from memory-mapped shards. Because the basis and the coarse quantizer are shared, scores from different shards are comparable and a global top-k is the exact merge of per-shard top-ks. That property is what makes the measurement below exact rather than approximate, and the companion paper [3] draws the arrangement.

Corpus. The corpus is defined by a seed and regenerated on demand, so the distributed build moves no corpus bytes. Each global shard draws a fresh rank-16 Gaussian basis, projects Gaussian coefficients through it with a linear decay of singular values from 1.0 to 0.3, and adds five percent noise, in an ambient dimension of 32. Its intrinsic dimension measures 15.4 to 17.0 at every size we tested, with no trend in row count. Real embeddings such as Cohere Embed v3 at 1024 dimensions measure 52 to 61 by the same estimator. Hubness, the concentration of nearest-neighbour lists on a few points [15], rises with corpus size on this recipe, at about 0.42 per decade at $k = 10$ against roughly 0.25 per decade on real data. We attribute the rise to how shards are added. Each shard draws its own random 16-dimensional subspace, and two random 16-dimensional subspaces of a 32-dimensional space are not orthogonal, so the rows of a new shard land close to rows of shards already present. Every added shard therefore adds rows near existing ones, and the rows that sit where many subspaces pass close together appear in more and more neighbour lists. Whatever its cause, the rise does not act on the quantity measured here. Recall against the exact scan depends only on whether the router reaches the cell that holds each reference neighbour (Section 4.3), and hubness changes which rows are the neighbours without changing that condition.

Build. The 500 servers were built between 2026-08-04 and 2026-09-24 by one job per server, each computing its 400 shards from the seed, so the index can be rebuilt and checked from the seeds alone. The build, its storage and its job shape are described in the companion paper [3].

Scale points. The same protocol has been run at 10^8 rows on 4 servers, 10^9 on 4, 10^{10} on 8, 10^{11} on 50 and now 10^{12} on 500. Per-server work is constant from 10^{11} on, two billion rows, so the step from 50 to 500 servers changes nothing per node and changes only the number of chances to draw a slow or failing one.

3 PROTOCOL

The measurement has four phases, each a pool of independent jobs on the cluster's batch system, every submission through a controller that paces submissions, caps concurrency at twenty and yields while the cluster is busy. The query-cache phase is one job that derives the query set, the first 25 rows of each of four seeded shards, 100 queries, and writes it to the shared volume. Every server reads the queries from that file, so all see the same queries without regenerating anything. These queries are corpus rows, and Section 4.4 adds a set that is not. The reference phase is one job per server that scans that server's two billion rows exactly with the asymmetric distance computation and writes its top-10 per query with scores. The routed phase is one job per server that runs the inverted-file search at 32 and at 128 probes and writes the same shape of partial. The score phase is one job that merges the 500 reference partials into the exact global top-10, merges the routed partials the same way, and reports recall at ten of the routed set against the reference set, per probe width, averaged over queries. A second routed phase at 16, 64 and 256 probes, submitted the day the first score landed and scored two days later, a third run of all four phases with queries outside the index, and three metadata phases that read only the partials and the index headers are described with their results in Sections 4.3 and 4.4.

Every job is idempotent on the volume. A job whose output already exists prints that fact and exits, and outputs are written to a temporary name and renamed, so a job interrupted mid-write leaves nothing behind. This is the property every recovery rule of the pool relies on [3]. The score program has not changed since the 10^9 run. The two phase scripts changed at this run to fit the smallest job class [3], and none of the changes touch what is computed.

4 RESULTS

4.1 Recall across four orders of magnitude

Table 1 and Figure 1 give the number the run exists to produce. At 10^{12} rows, routed search at 128 probes of 2048 cells returns 0.999 of the exact scan's top ten, and at 32 probes 0.989. At 128 probes the figure is 0.999, 0.9988 and 0.9986 at the three smaller scales, a spread of 0.0004. At 32 probes it rises slowly with scale, 0.979, 0.982, 0.983, 0.989, and Section 4.2 traces that rise to the number of rows alone. The interim score, taken on the first 349 servers to finish both phases, gave 0.985 and 0.999, and recall at 128 probes on the first half of those 349 servers alone was also 0.999. The number is stable inside the run as well as across runs.

The mean hides little. At 32 probes 90 of the 100 queries have recall 1.0, nine have 0.9 and one has 0.8. At 128 probes 99 queries have recall 1.0 and one has 0.9, so the mean of 0.999 is one query missing one neighbour. Resampling the 100 queries gives 95 percent

Table 1: Recall at ten of routed search against the exact full scan of the same compressed index, by corpus size. Queries are 100 at 10^8, 10^9 and 10^{12} rows and 500 at 10^{10} and 10^{11}. The last column is the median over servers of the wall time of one server's exact scan. The 10^{12} jobs ran on one CPU each (152 exact scans at six, with the same median time), where the earlier runs had six, so that column is not comparable across rows (Section 4.5).

rows	servers	32 probes	128 probes	median scan (s)
10^8	4	0.979	0.999	47
10^9	4	see text		589
10^{10}	8	0.982	0.999	8495
10^{11}	50	0.983	0.999	19041
10^{12}	500	0.989	0.999	2664

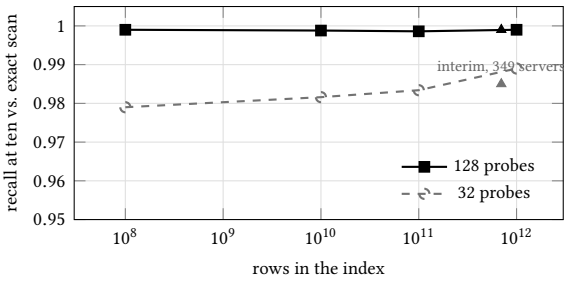


Figure 1: Recall against the exact scan by corpus size. The triangles are the interim score taken while the 10^{12} run was two thirds through its routed phase, on the 349 servers that had finished both phases, 698 billion rows. They sit on the final points.

Table 2: Recall at ten against the exact scan of a subset of k servers of the trillion-row index, the mean of five seeded random subsets of k servers (at $k = 500$, the whole index). Only the number of rows changes along k. The first k servers give values within the random subsets' spread. Eight of the twelve values of k are shown, and Figure 2 plots all twelve. Derived from the record by `figdata/fig_data.py`.

servers	rows	16 probes	32	64	128
1	2×10^9	0.797	0.905	0.965	0.991
5	10^{10}	0.846	0.931	0.979	0.996
10	2×10^{10}	0.861	0.943	0.984	0.997
50	10^{11}	0.906	0.970	0.990	0.998
100	2×10^{11}	0.913	0.971	0.993	0.999
200	4×10^{11}	0.942	0.984	0.996	1.000
300	6×10^{11}	0.948	0.988	0.997	0.999
500	10^{12}	0.952	0.989	0.997	0.999

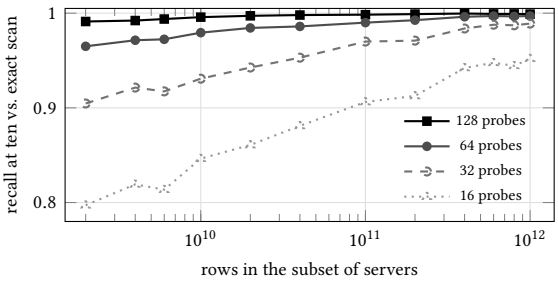


Figure 2: Recall against the exact scan of k servers of the one index, mean of five random subsets, from two billion to a trillion rows. Wide probes are flat, and narrow probes gain as rows are added.

bootstrap intervals of 0.982 to 0.995 at 32 probes and 0.997 to 1.000 at 128. The earlier scale points recorded only the mean, so they carry no interval.

The 10^9 row has no routed-versus-scan figure in the table because that run's purpose was different. It carried a cold store of the original float vectors, so it measured true recall against float ground truth. The routed shortlist at 128 probes returned 0.999 of the compressed scan there too, the compressed scan itself found only 0.592 of the true neighbours, and reranking the shortlist against the stored originals raised that to 0.991. That row is the reason Section 5 exists.

4.2 Recall inside one index as the rows grow

The four runs of Table 1 differ in more than size. Each was a separate index with its own bootstrap shard, coarse quantizer and queries, so a change between them could come from any of those. The trillion-row run removes that confound. The exact top ten over any subset of servers is the merge of that subset's partials, and so is the routed top ten, so recall against the exact scan of any subset is exact and needs no new scan. A subset of k servers is a corpus of $2k$ billion rows from the same generator, with the same basis, coarse quantizer, router and queries, and along k only the number of rows changes. We took the first k servers and five seeded random subsets of k servers for twelve values of k from 1 to 500. The random subsets hold at most one of the four servers the queries came from.

Table 2 and Figure 2 give the result. At 128 probes recall moves from 0.991 at two billion rows to 0.999 at a trillion, and at 256 probes from 0.999 to 1.000. At narrow widths recall rises with the number of rows, at 16 probes from 0.797 to 0.952 and at 32 probes from 0.905 to 0.989. The first k servers give the same curve within the random subsets' spread, so the rise does not come from the servers that hold the queries' home shards. Growing the corpus from two billion to a trillion rows therefore causes no measurable loss of routed recall at any width under this protocol, and at narrow widths it raises it. The rise at 32 probes across the separate runs, 0.979, 0.982, 0.983 and 0.989, is the same effect with the confound present.

4.3 Recall from the probe order alone

Every reference neighbour lives in one of the 2048 cells, and each query orders the cells by the bound the router uses. Recall at width p is then the share of reference neighbours whose cell has rank below p in its query's order, because inside a probed cell the routed pass scores with the same asymmetric distance the exact scan used, so a reference neighbour is returned exactly when its cell was probed. That identity, stated over finite sets with the in-cell

Table 3: Recall at ten predicted from the cell ranks of the reference neighbours, and recall measured by a routed pass at that width, 10^{12} rows, 100 queries. The 32 and 128 columns were measured first and checked against the prediction. The 16, 64 and 256 columns were predicted and recorded before the sweep that measured them was submitted. The last row counts the queries whose predicted recall is 1.0.

probes	1	2	4	8	16	32	64	128	256
predicted	0.406	0.599	0.727	0.867	0.952	0.989	0.997	0.999	1.000
measured					0.952	0.989	0.997	0.999	1.000
queries with recall 1.0	1	4	15	37	68	90	97	99	100

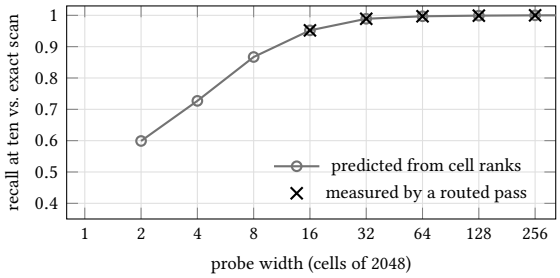


Figure 3: The recall curve over probe width computed from the cell rank of each reference neighbour (line, nine widths from the census merge, no distance scan) and the five widths measured by routed passes over the whole index (crosses). The crosses sit on the line at every width.

condition as its hypothesis, is checked in Lean 4 with Mathlib in the repository (`lean/Pvldb1t/Reachability.lean`, theorem `recall_eq_share`). The share needs the cell of each of the 1000 reference neighbours and nothing else, so it was computed from the index metadata in 500 jobs of half a minute each and one merge, with no distance scan. Table 3 and Figure 3 give it at nine widths. It agreed with the two widths already measured, 32 and 128, to the last digit. We then recorded the values at 16, 64 and 256, submitted a routed pass at those three widths on all 500 servers, and the measured recall came back 0.952, 0.997 and 1.000, again equal to the prediction at every width. The neighbours' cell ranks have median 1, ninetieth percentile 9, ninety-ninth percentile 32 and maximum 169, so the single neighbour that 128 probes miss sits in the 170th cell of its query's order, and 256 probes return every neighbour of every query.

The identity also explains the stability across scale. The corpus grows by adding shards under one shared coarse quantizer, so an added shard puts rows into the same 2048 cells and leaves every query's probe order unchanged. What changes is which rows are the ten nearest, and recall at width p changes only if those rows move to cells of different rank. More rows in every cell put the nearest ten closer to the query, and nearer rows sit in cells earlier in its probe order, which is the rise at narrow widths that Section 4.2 measures. The agreement says that on this index recall is a property of the coarse quantizer's probe order and of nothing downstream of it. It also gives a cheap instrument. Once the reference neighbours are known, the recall curve at any width is a count over cell ranks, and a routed pass at a new width confirms a number that was already on the page.

4.4 The cells, query placement, and queries outside the index

The same 500 metadata jobs took a census of the cells. The 2048 cells hold between 49.2 million and 2.15 billion rows, with mean 488 million and median 418 million, a Gini coefficient of 0.328, the largest one percent of cells holding 3.7 percent of rows, and no empty cell. The imbalance is the shared coarse quantizer's, fitted once on the bootstrap shard, and it is what the router's probe order works against.

The 1000 reference neighbours of the 100 queries sit on 233 of the 500 servers, and one server holds 16.6 percent of them, because the queries are rows of four seeded shards and each shard's own basis keeps a query's neighbours near its home shard. Losing one server at random would cost 0.002 of recall in expectation, and losing that one server would cost 0.166. The same recipe is used at every scale point, the first rows of four shards spread evenly across the index's shard range, 25 rows each at 10^8 and 10^{12} rows and 125 each at 10^{10} and 10^{11}. The trend across sizes is therefore measured with the query distribution fixed. Because every query is a corpus row, it is also its own nearest neighbour, and its neighbours lie near its home shard.

A second measurement over the same index removes both. Its 100 queries are the first 25 rows of each of shards 200000, 250000, 300000 and 350000 of the same generator, seeds no corpus shard uses, so no query is in the index and none has a home server. The reference scan, the routed passes at 32 and 128 probes and the exact merge were rerun over all 500 servers with the programs of the first run. The 1000 reference neighbours of these queries sit on 404 of the 500 servers, no server holds more than 1.0 percent of them, and the median query has its ten neighbours on eight servers. Routed recall against the exact scan was 0.969 at 32 probes and 0.999 at 128, with 95 percent bootstrap intervals over queries of 0.954 to 0.982 and 0.997 to 1.000, against 0.989 and 0.999 for the corpus-row queries. At 32 probes 78 of the 100 queries had recall 1.0 and the lowest had 0.6. At 128 probes the two query sets agree. At 32 probes the queries the index does not contain lose 0.020, which we attribute to the two advantages a corpus row has and this design does not separate, its own row among its ten neighbours and neighbours in the cells its own shard's basis favours, which the shared coarse quantizer reaches early. For the usual protocol, in which queries are not rows of the database, 0.969 and 0.999 are the figures to quote.

4.5 Cost

The exact scan of one server's two billion rows at one CPU took 44 minutes at the median server, and a routed pass 15 to 27 minutes depending on width. Those times include pulling the image, installing dependencies, cloning the repository and opening 400 shards from network block storage, so they are the cost of the measurement job and not of inverted-file search, and the ratio between them is not a latency figure for the method. The first run, the exact scan and the routed passes at 32 and 128 probes, consumed 772 CPU-hours, and the second sweep at 16, 64 and 256 probes consumed 577, 1349

CPU-hours for the corpus-row queries. The run with queries the index does not contain consumed 835 more. Its exact scan does the same work for any query set of the same size and took 437 CPU-hours, against 418 in the first run. The per-phase distribution of wall time and the behaviour of the pool are in the companion paper [3].

5 WHAT THE RESULT DOES AND DOES NOT SHOW

Recall of the routed shortlist against the exact scan of the same compressed index does not degrade from 10^8 to 10^{12} rows. That is a statement about the inverted-file layer, its shared coarse quantizer and its merge. Growing the corpus by adding shards causes no measurable loss of routed recall against the exact scan at any width measured, at narrow widths it raises it, and Section 4.3 says why. These statements are for corpus-row queries, since queries outside the index were measured at 10^{12} rows only. Table 4 lists what could make the result wrong or narrower than it reads, and what in the record addresses each.

It is not a statement about recall against the truth. The compressed scan at 10^9 rows found 0.592 of the float neighbours, and on a real corpus of 15 million Cohere Embed v3 vectors the same index found 0.693 of them by exact scan, 0.689 by routing at 256 probes, and 0.981 after reranking the routed shortlist against stored originals. On that real corpus the routed shortlist returned 0.990 of the compressed scan at 256 probes, so the routing result transfers, and the compression result does not. Four bits per dimension at this rate loses neighbours that only a rerank tier recovers, and the trillion-row corpus, at intrinsic dimension 16 against a real embedding's 52 to 61, cannot show that. It is an easy corpus for the code and a hard one only for the hubness the shards' overlap induces.

A cold store of originals at 10^{12} rows would be 128 TB of float vectors, so true recall and reranking remain established at 10^9 and at the 15-million-row real pilot, not here. The seeded corpus does allow one bound. The float rows named by a shortlist can be regenerated from the seed, so the ADC top-10 of each query can be reranked by float cosine within the shortlist that contains it, and the share of the ADC top-10 that survives that rerank is an upper bound on ADC-only true recall, because a corpus float neighbour that lies in the shortlist is a shortlist float neighbour. The set inequality behind the bound is checked in Lean 4 with Mathlib in the repository (`lean/Pvldb1t/RerankBound.lean`, theorem `rerank_upper_bound`). Table 5 gives the result on 11002 regenerated rows. Of the reference scan's ADC top-10, 0.621 survives the float rerank of its own shortlist on average, with 22 of 100 queries keeping all ten and a minimum of 0.100, so ADC-only true recall at 10^{12} rows is at most 0.621 on this corpus. The routed shortlists behave the same way, 0.623 at 16 probes and 0.621 at 256. At one billion rows the cold store gave ADC-only true recall of 0.592, inside this bound. After reranking, the routed top-10 agrees with the reranked reference top-10 at 0.940 for 16 probes, 0.982 for 32, 0.996 for 64 and 1.000 from 128 on, within 0.012 of the recall before reranking at every width and equal to it at 256 probes, so the rerank neither hides nor amplifies what routing costs. The mean cosine of the ADC top-10 to the query is 0.863 and of the reranked top-10 0.929. Each query is itself a corpus row, and that row is in the ADC top-10 for 100 of 100 queries and in the reranked top-10 for 100 of 100.

Each query set has one hundred queries, which is few. The exact scan is compute-bound in the query count and 500 queries would have cost about five times the 418 CPU-hours. The queries outside the index, whose neighbours spread over 404 servers, show that the concentration of reference neighbours on one server in Section 4.4 is a property of the query set and not of the index.

6 RELATED WORK

Product quantization with asymmetric distance computation and inverted files [10] is the basis of the index, and the two-stage arrangement in which a compressed scan produces a shortlist that a finer code or the original vectors rerank dates from the same group's billion-vector work [11]. Locally optimized product quantization [13] learns a separate rotation and codebook for each cell, the opposite choice to the shared basis we use to keep scores comparable across shards, and anisotropic quantization [8] and RaBitQ [6] are the current forms of the quantizer with, in the latter case, an error bound on the estimated distance. Locally adaptive scalar quantization [2] puts a compressed representation under a graph index and reranks from a second tier, which is the rerank tier our 10^9 and real-embedding figures use. Locality-sensitive hashing [9] and navigable graphs [14] are the usual alternatives to the inverted file and are not what this paper measures.

Billion-scale practice is established. Faiss [5, 12] serves it on one machine or one GPU, DiskANN [18] and SPANN [4] on a single node with the index on disk, and the NeurIPS'21 Big ANN competition [17] fixed the datasets and the reporting for that scale, with its 2023 successor [16] turning to filtered, sparse, out-of-distribution and streaming search at ten million rows. Distributed vector databases such as Milvus [19] and its cloud-native successor [7] shard an index across nodes and merge per-node results, which is the arrangement our fleet uses without a serving layer, and recent work places a billion-scale index on distributed storage [21]. A single graph index has been spread over more than a thousand machines at 5×10^{10} vectors [1]. We found no published measurement of a routed index against the exact scan of the same index at 10^{12} rows. This paper adds that measurement and the identity that predicts it from the router's probe order. The companion paper [3] describes how it was obtained on a shared cluster.

7 REPRODUCIBILITY

The repository snapshot at the availability URL, archived as DOI 10.5281/zenodo.23608470, carries the corpus seeds and generator, the build, scan, routed, score and metadata scripts, the pool driver, and under `benchmarks/fleet/record/1t/` the pool state files, every driver log, the pilot logs, the interim score and the score logs whose RESULT_JSON lines hold every per-server wall time, those of the run with queries outside the index included. The figures and tables are produced from those logs by `fig_data.py` in `paper/pvldb1t/figdata`, and the per-query figures for the queries outside the index by `fleet_nonmember_analysis.py` in `benchmarks/fleet`. The shared volume of the run, with the bootstrap shard, the coarse quantizer and every per-server partial result, and the partials and per-volume index checksums of the run

Table 4: Threats to validity and what addresses each.

threat	what addresses it
The corpus is synthetic, intrinsic dimension about 16 against 52 to 61 for real embeddings	On 15 million Cohere Embed v3 vectors the routed shortlist returned 0.990 of the compressed scan at 256 probes, so the routing result transfers. The compression result does not, and this table's next row bounds it.
The reference is the compressed index's own exact scan, not float ground truth	At 10^9 rows the compressed scan found 0.592 of the float neighbours. At 10^{12} the rerank bound puts the compressed ranking at no more than 0.621, and after reranking the routed top ten agrees with the reranked reference at 0.982 for 32 probes and 1.000 from 128.
Each query is a corpus row, so it is its own nearest neighbour	Every query finds itself in its top ten, so the self-match is one of the ten neighbours routing must reach. On 100 queries that are not in the index, recall was 0.969 at 32 probes and 0.999 at 128 (Section 4.4).
Queries come from four shards and their neighbours are concentrated, 16.6 percent on one server	The random subsets of Section 4.2 hold at most one query home server and match the first k servers. The queries that are not in the index put their neighbours on 404 servers, no more than 1.0 percent on any one, and recall at 128 probes did not change, while at 32 probes it fell by 0.020 (Section 4.4).
One hundred queries per set	Bootstrap intervals over queries of 0.982 to 0.995 at 32 probes and 0.997 to 1.000 at 128 for the corpus rows, and 0.954 to 0.982 and 0.997 to 1.000 for the queries outside the index.
Scale is confounded with the index across runs	The nested series varies only the number of rows inside one index (Section 4.2).
Code changed during the run	The merge and score code did not change, and the phase scripts came from a configuration map fixed for each run.

Table 5: Rerank bound on regenerated float rows, 100 queries, shortlists of width 100 cut from the merge of per-server top-10 partials. Survival is the share of the ADC top-10 in the float top-10 of the same shortlist, an upper bound on ADC-only true recall. Transfer is the share of the routed shortlist's float top-10 that is in the reference shortlist's float top-10.

shortlist	ref.	16	32	64	128	256
survival of the ADC top-10	0.621	0.623	0.623	0.624	0.622	0.621
transfer after rerank	–	0.940	0.982	0.996	1.000	1.000

with queries outside the index are archived as DOI 10.5281/zenodo.23608434, each file with its sha256. The Lean development under `paper/pvldb1t/lean/` builds against Mathlib at Lean 4.32.2, and its axiom audit, recorded in `AXIOMS.txt`, shows every theorem depending on propositional extensionality, quotient soundness and choice only. The twelve repository commits that job containers cloned during the first run are listed in the result note `docs/notes/1T_RECALL_RESULT_2026-09-27.md`, and the configuration map that carried the phase scripts is recorded by content hash. Rebuilding the index from seed costs about three and a half hours per server at the pool's width.

8 CONCLUSION

A trillion-row compressed index was scanned exactly and searched by routing over the same shards. Scaling from 10^8 to 10^{12} rows causes no measurable loss of routed recall against the exact scan for corpus-row queries under this protocol, inside the one index recall at narrow widths rises with the number of rows, and the recall at any probe width is the share of reference neighbours whose cell the router reaches, a count that predicted three measured widths to the last digit. Against float rows regenerated from the seed, the compressed ranking keeps at most 0.621 of the true neighbours, and after a rerank the routed top ten agrees with the reranked reference at 0.982 for 32 probes and 1.000 from 128 on. Queries the index does not contain give 0.969 at 32 probes and 0.999 at 128, the figures for the usual protocol. The measurements with corpus-row queries cost 1349 CPU-hours and about four days, and the one with queries outside the index 835 CPU-hours more. All of it is reproducible from the repository, which carries the seeds, the scripts, the driver, the state files, the driver logs, the pilots, the interim score and the final scores with every per-server wall time.

ACKNOWLEDGMENTS

This work used the National Research Platform's Nautilus cluster. The National Research Platform is supported in part by National Science Foundation awards CNS-1730158, ACI-1540112, ACI-1541349, OAC-1826967, OAC-2112167, CNS-2100237 and CNS-2120019. We thank its operators and community.

REFERENCES

[1] Philip Adams, Menghao Li, Shi Zhang, Li Tan, Qi Chen, Mingqin Li, Zengzhong Li, Knut Risvik, and Harsha Vardhan Simhadri. 2025. DistributedANN: Efficient Scaling of a Single DiskANN Graph Across Thousands of Computers. *arXiv preprint arXiv:2509.06046* (2025). https://arxiv.org/abs/2509.06046

[2] Cecilia Aguerrebere, Ishwar Singh Bhati, Mark Hildebrand, Mariano Tepper, and Theodore Willke. 2023. Similarity Search in the Blink of an Eye with Compressed Indices. *Proceedings of the VLDB Endowment* 16, 11 (2023), 3433–3446. https://doi.org/10.14778/3611479.3611537

[3] Andrew H. Bond. 2026. A Trillion Vectors in the Smallest Job Class [Scalable Data Science]. Under submission to PVLDB.

[4] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. Curran Associates, Inc., Red Hook, NY, USA.

[5] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Transactions on Big Data* 12, 2 (2026), 346–361. https://doi.org/10.1109/TBDATA.2025.3618474

[6] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 3 (2024). https://doi.org/10.1145/3654970

[7] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: A Cloud Native Vector Database Management System. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3548–3561. https://doi.org/10.14778/3554821.3554843

[8] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research)*, Vol. 119. PMLR, 3887–3896. https://proceedings.mlr.press/v119/guo20h.html

[9] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, New York, NY, USA, 604–613. https://doi.org/10.1145/276698.276876

[10] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. https://doi.org/10.1109/TPAMI.2010.57

[11] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in One Billion Vectors: Re-rank with Source Coding. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 861–864. https://doi.org/10.1109/ICASSP.2011.5946540

[12] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. https://doi.org/10.1109/TBDATA.2019.2921572

[13] Yannis Kalantidis and Yannis Avrithis. 2014. Locally Optimized Product Quantization for Approximate Nearest Neighbor Search. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2329–2336. https://doi.org/10.1109/CVPR.2014.298

[14] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

[15] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. 2010. Hubs in Space: Popular Nearest Neighbors in High-Dimensional Data. *Journal of Machine Learning Research* 11, 86 (2010), 2487–2531. https://jmlr.org/papers/v11/radovanovic10a.html

[16] Harsha Vardhan Simhadri, Martin Aumüller, Matthijs Douze, Dmitry Baranchuk, Amir Ingber, Edo Liberty, George Williams, Ben Landrum, Magdalen Manohar, Mazin Karjikar, Laxman Dhulipala, Meng Chen, Yue Chen, Rui Ma, Kai Zhang, Yuzheng Cai, Jiayang Shi, Weiguo Zheng, Yizhuo Chen, Jie Yin, and Ben Huang. 2025. Results of the Big ANN: NeurIPS'23 Competition. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, Vol. 38. Curran Associates, Inc., Red Hook, NY, USA, 76394–76408. https://doi.org/10.52202/085713-2303

[17] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2022. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track (Proceedings of Machine Learning Research)*, Vol. 176. PMLR, 177–189. https://proceedings.mlr.press/v176/simhadri22a.html

[18] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32. Curran Associates, Inc., Red Hook, NY, USA.

[19] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. ACM, New York, NY, USA, 2614–2627. https://doi.org/10.1145/3448016.3457550

[20] Derek Weitzel, Ashton Graves, Sam Albin, Huijun Zhu, Frank Würthwein, Mahidhar Tatineni, Dmitry Mishin, Elham E. Khoda, Mohammad Firas Sada, Larry Smarr, Thomas DeFanti, and John Graham. 2025. The National Research Platform: Stretched, Multi-Tenant, Scientific Kubernetes Cluster. In *Practice and Experience in Advanced Research Computing 2025: The Power of Collaboration (PEARC '25)*. ACM, New York, NY, USA. https://doi.org/10.1145/3708035.3736060

[21] Kun Yu, Jiabao Jin, Xiaoyao Zhong, Peng Cheng, Lei Chen, Zhitao Shen, Jingkuan Song, Hengtao Shen, and Xuemin Lin. 2025. Approximate Nearest Neighbor Search of Large Scale Vectors on Distributed Storage. *arXiv preprint arXiv:2510.17326* (2025). https://arxiv.org/abs/2510.17326